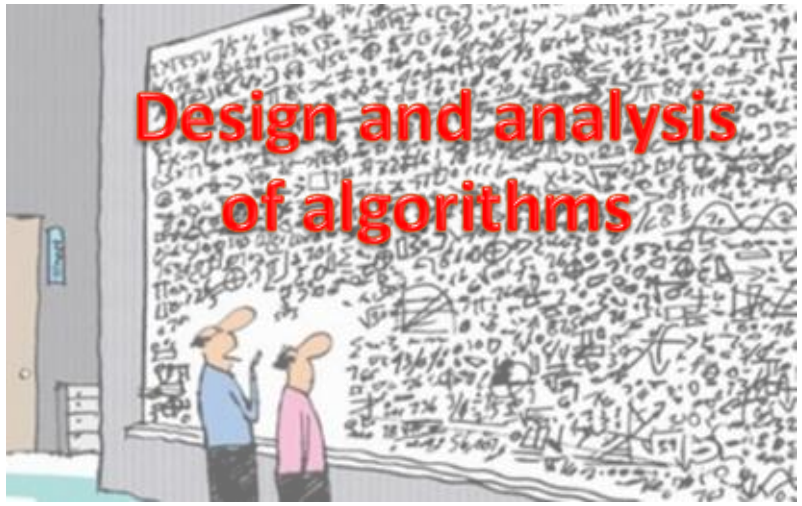


- **Bloom filters**

Objective

no need to store actual elements

- Given a set of elements S, we want a data structure for supporting insertions and querying about membership in S.
  - Minimize the use of memory: use a single bit per element.
  - Can check membership as fast as possible.
  - Probabilistic test.
- Burton Bloom: The Bloom filter data structure. Comm. ACM, July 1970.
- http://www.eecs.harvard.edu/~michaelm/NEWWORK/postscripts/BloomFilterSurvey.pdf

Example

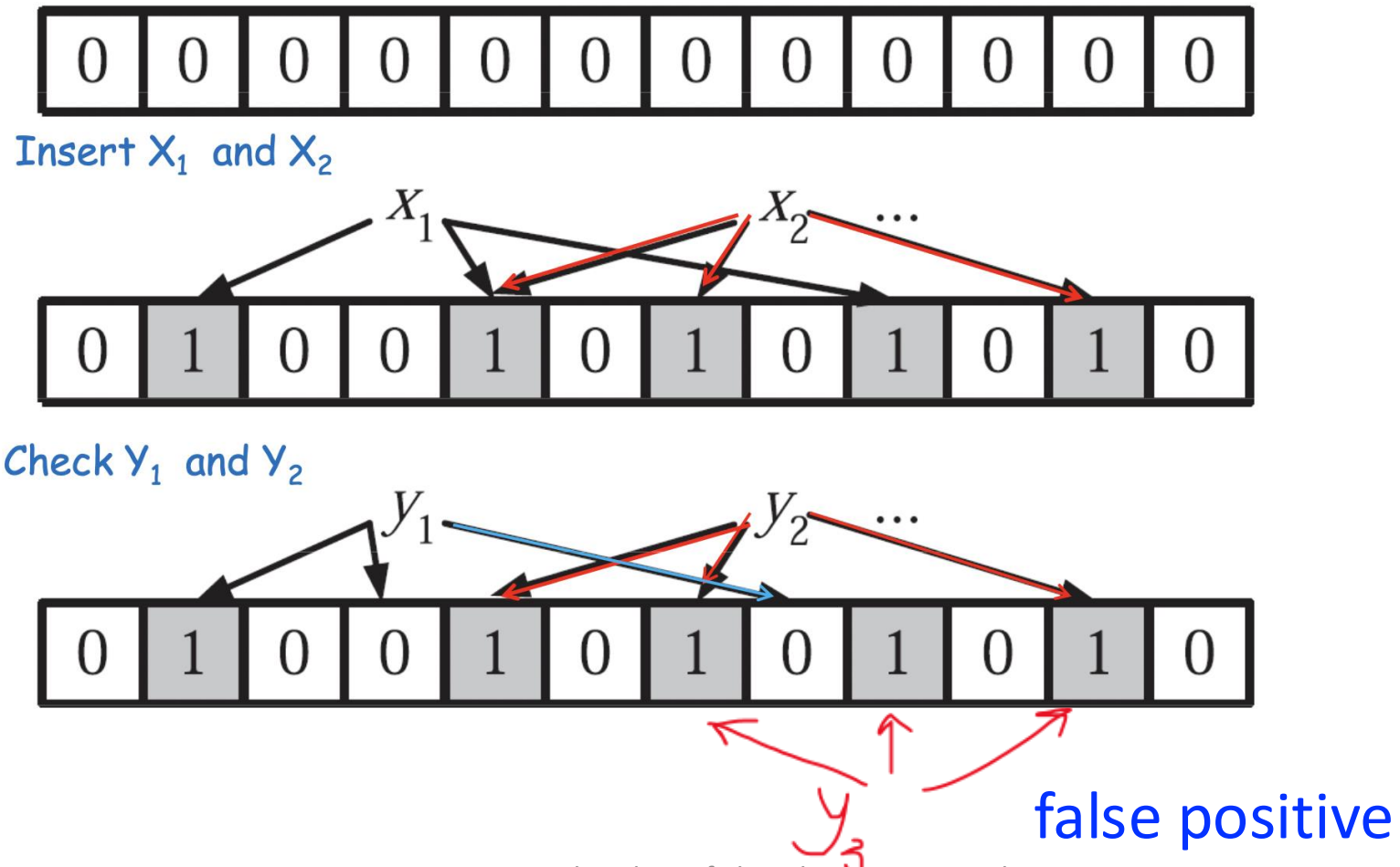
- We have a set S of 10^9 e-mail addresses, where the typical e-mail address is 20 bytes.
  - Requires 20 Gbytes of memory. Too expensive to store in main memory.
- We can afford 1 Gbyte of memory, which is approximately 10^9 bytes or 8×10^9 bits.
- How can we put S in main memory to query it?

The bloom filter

- A Bloom filter is an array T of m bits representing a set $S = \{x_1, \dots, x_n\}$ of n elements.
  - T set to 0 initially.
- k independent hash functions $h_1, \dots, h_k$ with range $\{1, 2, \dots, m\}$.
  - Assume that each hash function maps each element in the universe to a random number uniformly over the range $\{1, 2, \dots, m\}$.
- For each element x in S, the bit $h_i(x)$ in the array is set to 1, for $1 \leq i \leq k$.
  - A bit in the array may be set to 1 multiple times for different elements.

SUHA

A bloom filter example, 3 hash functions



Standard Bloom filter

- To check membership of y in S, check whether $h_i(y) = 1, \forall i = 1, \ldots, k$.
  - If not, y is definitely not in S (no false-negative).
  - If yes, we conclude that $y \in S$, but sometimes this conclusion is wrong (we allow for false-positive).
- For many applications, false positives are acceptable as long as the probability of a false positive is small enough.
- Note that no deletions are allowed (otherwise we may get false-negatives).
- We will assume that $kn < m$.

since the corresponding bits were set to 1 because of other elements

because we might set to 0 a bit that was set to 1 by some other element

otherwise, the array will have mostly 1s and search(x) will always return true

Probabilities (false positives)

- After all n members of S have been hashed to a Bloom filter, the probability that a specific bit is still 0 is

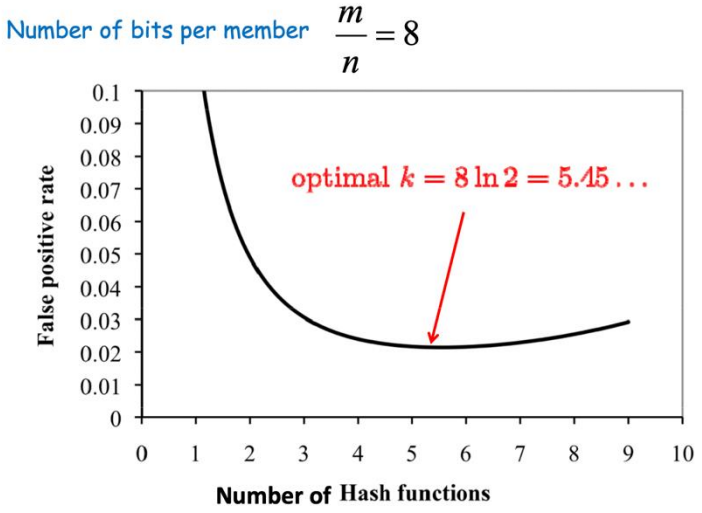
$$p = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-\frac{kn}{m}}.$$

Prob. a specific bit = 0 using a single h is $\left(1 - \frac{1}{m}\right)^{n} \approx e^{-\frac{n}{m}}$

- For a non-member, it may be found to be a member of S (all its k bits are nonzero) with false-positive probability

$$f = (1 - p)^k = (1 - e^{-\frac{kn}{m}})^k.$$

Two competing forces k, k as $k \uparrow$



Simple algebra: $\min_k f(k) \rightarrow k_{opt} = \frac{m}{n} \ln 2.$

$$f_{opt} = (0.5)^{\frac{m}{n} \ln 2} \approx (0.6185)^{\frac{m}{n}}.$$

Counting Bloom filters

To check membership of y in S, check whether $h_i(y) > 0, \quad \forall i = 1, \ldots, k$

- Initially proposed for distributed caching.
- Every entry in a counting Bloom filter is a small counter (rather than a single bit).
  - When an item is inserted into the set, the corresponding counters are each incremented by 1.
  - When an item is deleted from the set, the corresponding counters are each decremented by 1.
- To avoid counter overflow, its size must be sufficiently large. It was found in practice that 4 bits per counter are enough.

When a counter does overflow, it may be left at its maximum value. Problem for false negatives!

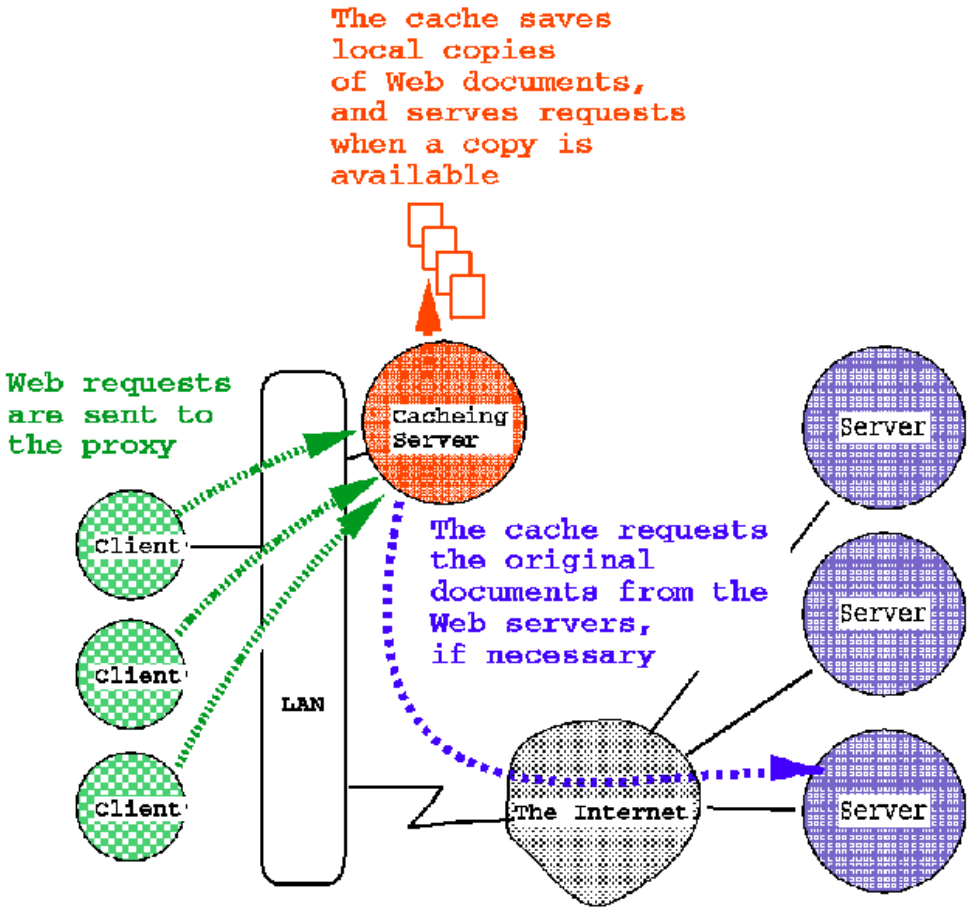
Some Bloom filter applications

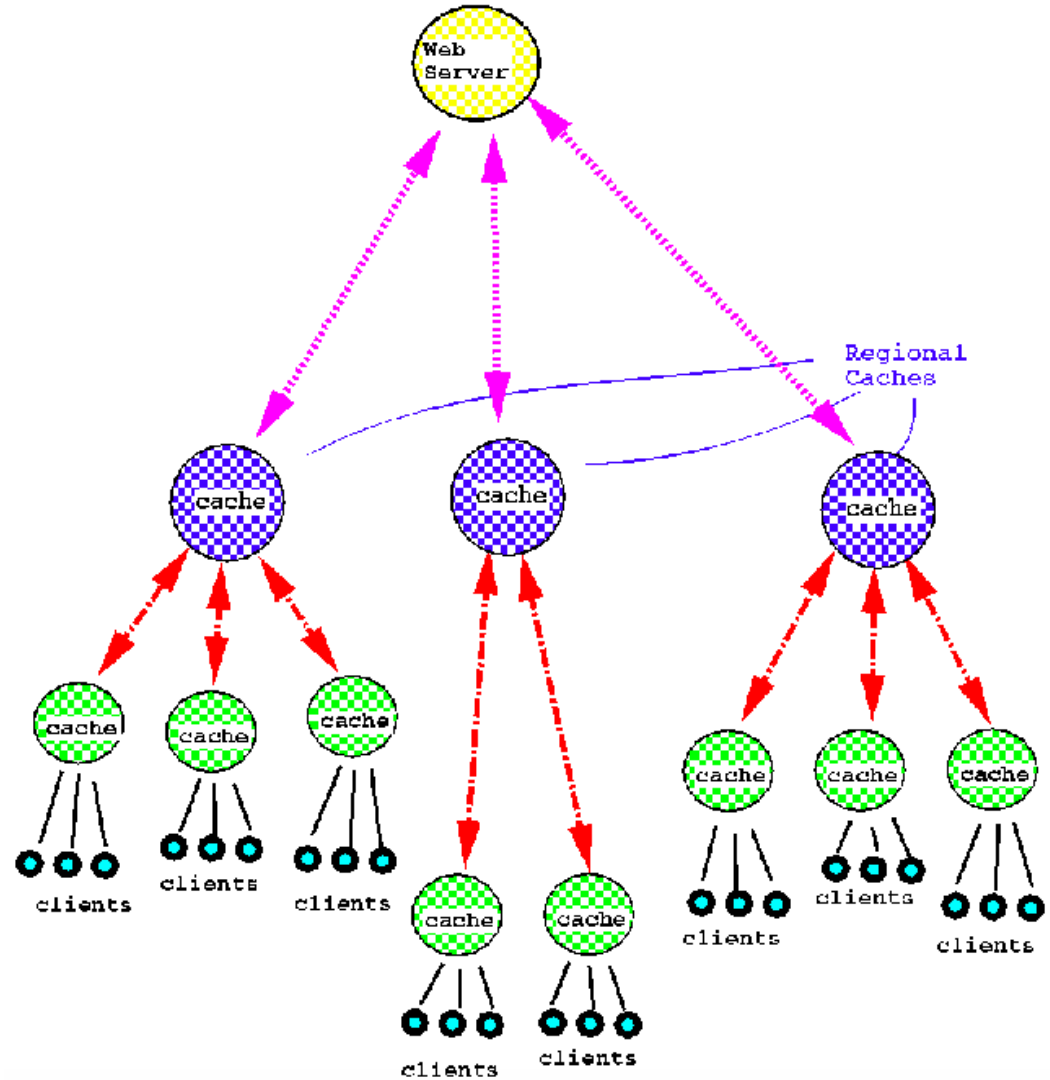
- Use the fact that Bloom filters allow for easy characterization of sets (decide set membership).
- To disclose my set of resources I can send my bloom filter! (the bit array + hash functions).
- Objects: resources, files, network addresses.
- Can also store counting information.
- Many interesting application areas in which: low cost of false positives, high cost for false negatives.

Distributed caching (of web pages)

this is how the Cloud works!

- On a cache miss a proxy server needs to determine if another proxy cache holds the desired Web page (instead of trying to obtain the page from the Internet).
- Proxies must know the content of other proxy caches.
- They send periodically Bloom filters that represent the content of their cache, instead of URL lists.
- Each cache maintains a counting Bloom filter (because of dynamic content), broadcasts periodically a simple Bloom filter to other proxies.
- Possible false positives and false negatives (why?).


The cache saves
local copies
of Web documents,
and serves requests
when a copy is
available
Web requests
are sent to
the proxy
Cacheing
Server
Client
Client
Client
LAN
The cache requests
the original
documents from the
Web servers,
if necessary
The Internet
Server
Server
Server


Web
Server
Regional
Caches
cache
cache
cache
cache
cache
cache
cache
cache
cache
cache
cache
cache
clients
clients
clients
clients
clients
clients
clients
clients

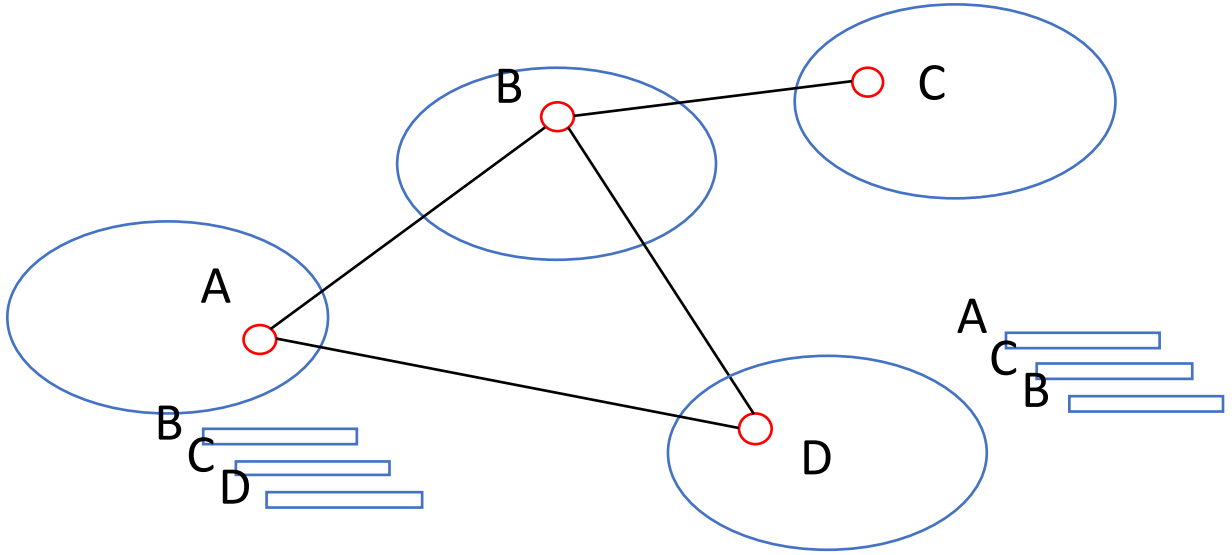
P2P: finding location of files

- Problem: who has which pieces of a large file.
- Use a bloom filter to characterize the objects available in location *a* (instead of the list of full ids of the objects).
  - False positives: creates extra requests and need to have alternative method to locate the item elsewhere if not available.
- Set reconciliation: find and send $S_a - S_b$.
  - False negatives: not all members of the $S_a - S_b$ are sent.
- Could be used in large *file distribution* systems.

I am **b** and ask **a** to send me the pieces of the file I don't have. I send her S_b. She computes $S_a - S_b$ and sends me the pieces in that set.

Network routing

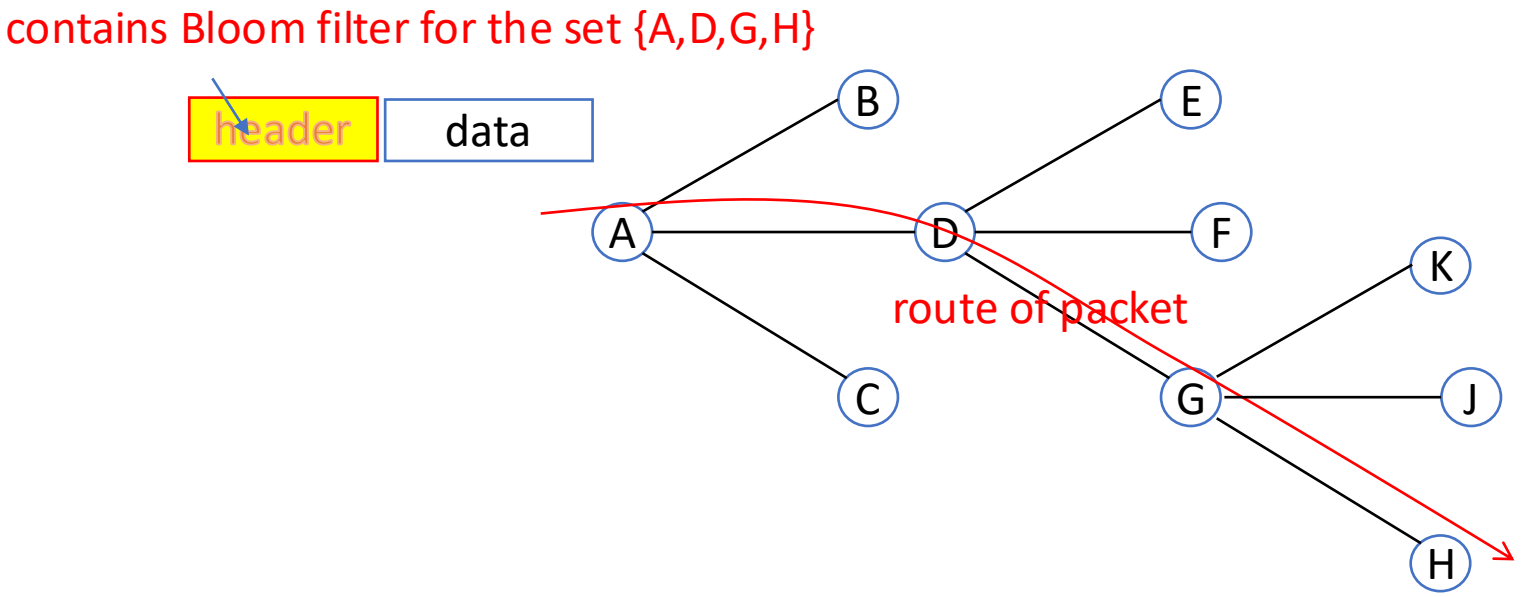
- Geographic routing:
  - Region is divided in areas with one responsible node
  - Each node has a bloom filter for reachability from itself and siblings
- False positive: extra path traversal



Packet routing

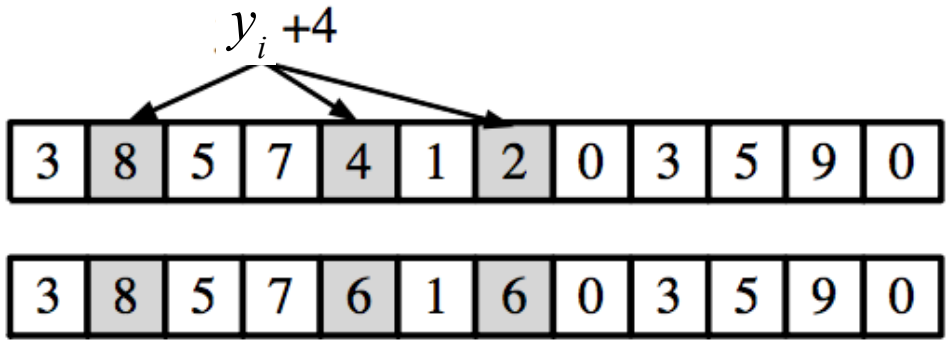
- Source routing
  - A packet stores its route (A,D,G,…) as a Bloom filter; each router uses this filter to find the next router in the path
- Detecting loops (unicast, multicast)
  - Use a bloom filter in the packet to mark nodes traversed

Router D that receives the packet checks which among its neighbors are in the route of the packet, i.e., in its Bloom filter. In this case it finds that G is in the route.

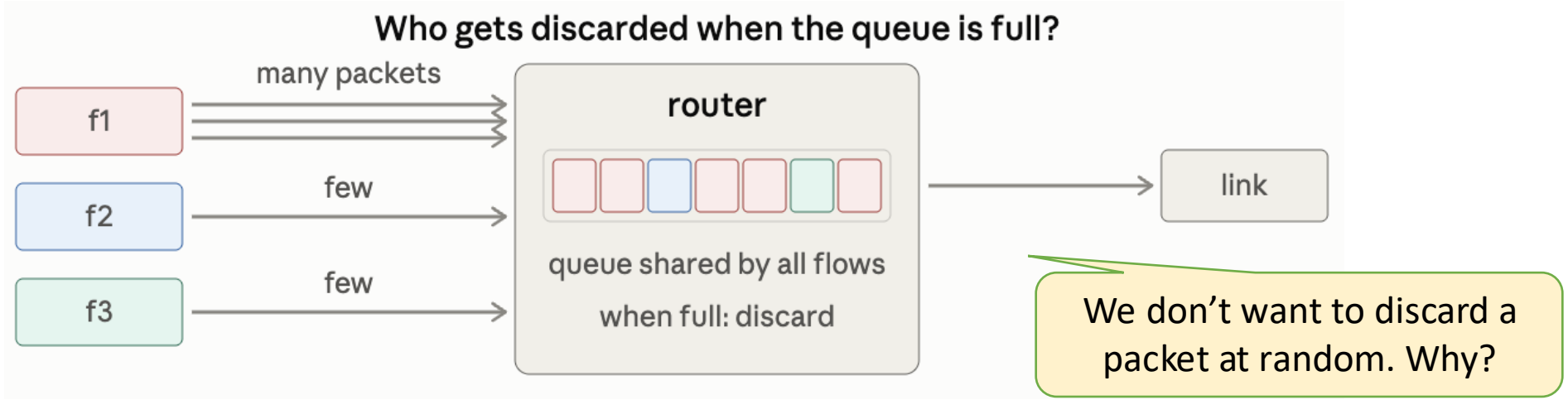


Recording heavy flows

- Use a counting Bloom filter, each location i stores byte volume v_i
  - When a packet *p* arrives, hash according to flow id *f*, add to all locations $i \in h(f)$ the volume of data *x* of the packet: $\forall i \in h(f), y_i = y_i + x$.
  - Estimate of the total volume carried by a flow $= \min_{i \in h(f)} \{y_i\}$.
- Conservative update.
  - Do not increase a location if it exceeds the value of the location with the minimum volume.



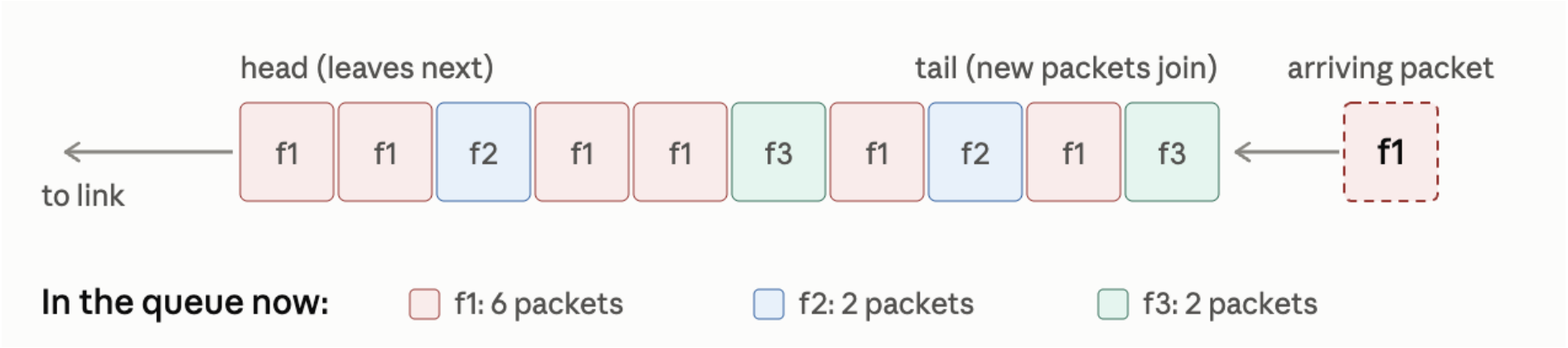
Fair queue management with a counting Bloom filter



- **Setting**: many flows share one router queue.
- **Problem**: when the queue is full and a packet arrives, whose packets do we discard?
- **Goal**: penalize flows that "contribute most to congestion": those that keep many packets in the queue for a long time.
- **Constraint**: millions of flows, little memory: no per-flow state.
- **Technique**: *mark packets with a probability that grows while their flow keeps the queue highly loaded; discard marked packets with priority*.
- **Tool**: a counting Bloom filter.

A router queue shared by many flows

- A flow f = (source, destination). Each box shows the flow of one queued packet.

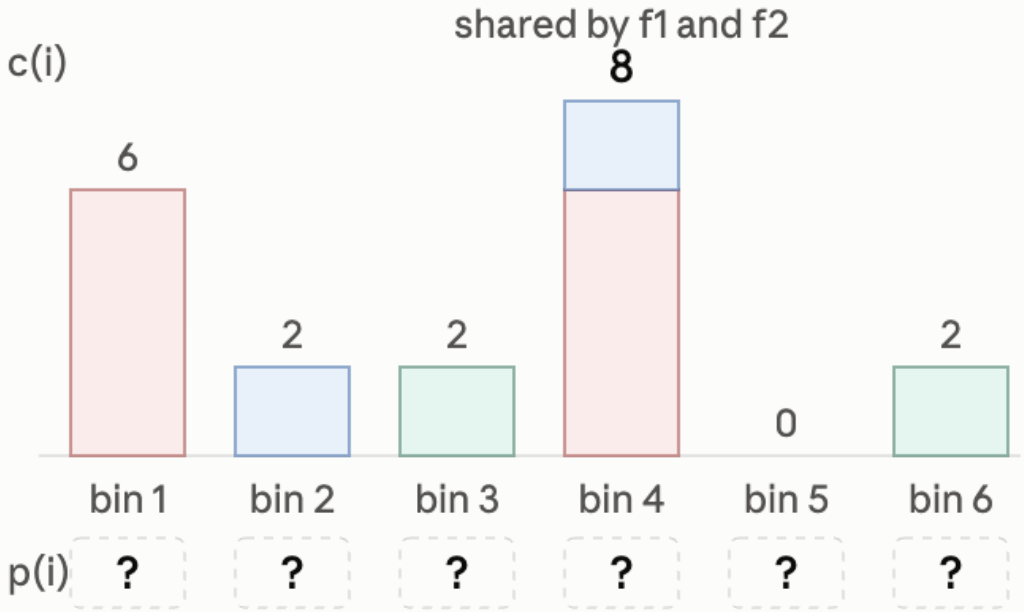


- When the queue is full, a *marked* packet is discarded to make room.
- **Goal**: mark packets of heavier flows with higher probability.
- **Obstacle**: millions of flows: we cannot keep one counter per flow.
- **Idea**: hash each flow into a small array of m counters (a counting Bloom filter).

Use a counting Bloom filter

Counting: m = 6 bins, k = 2 hash functions per flow

- f1: 6 packets in the queue, h(f1) = {1, 4}
- f2: 2 packets in the queue, h(f2) = {2, 4}
- f3: 2 packets in the queue, h(f3) = {3, 6}



Estimate of queued packets

est(f) = min of c(i) over the bins of f

est(f1) = min(6, 8) = 6	true 6
est(f2) = min(2, 8) = 2	true 2
est(f3) = min(2, 2) = 2	true 2

Bin 4 alone says 8 for f2.

The min picks the clean bin, so the collision does no harm.

Each bin i also keeps a marking probability p(i), used to mark packets.

How should p(i) be set over time?

How to update $p(i)$

When does p(i) change? Only when a packet arrives or departs

Arrival (packet of flow f)

for each bin i in h(f):
c(i) up by 1;
if c(i) > t, p(i) up by δ

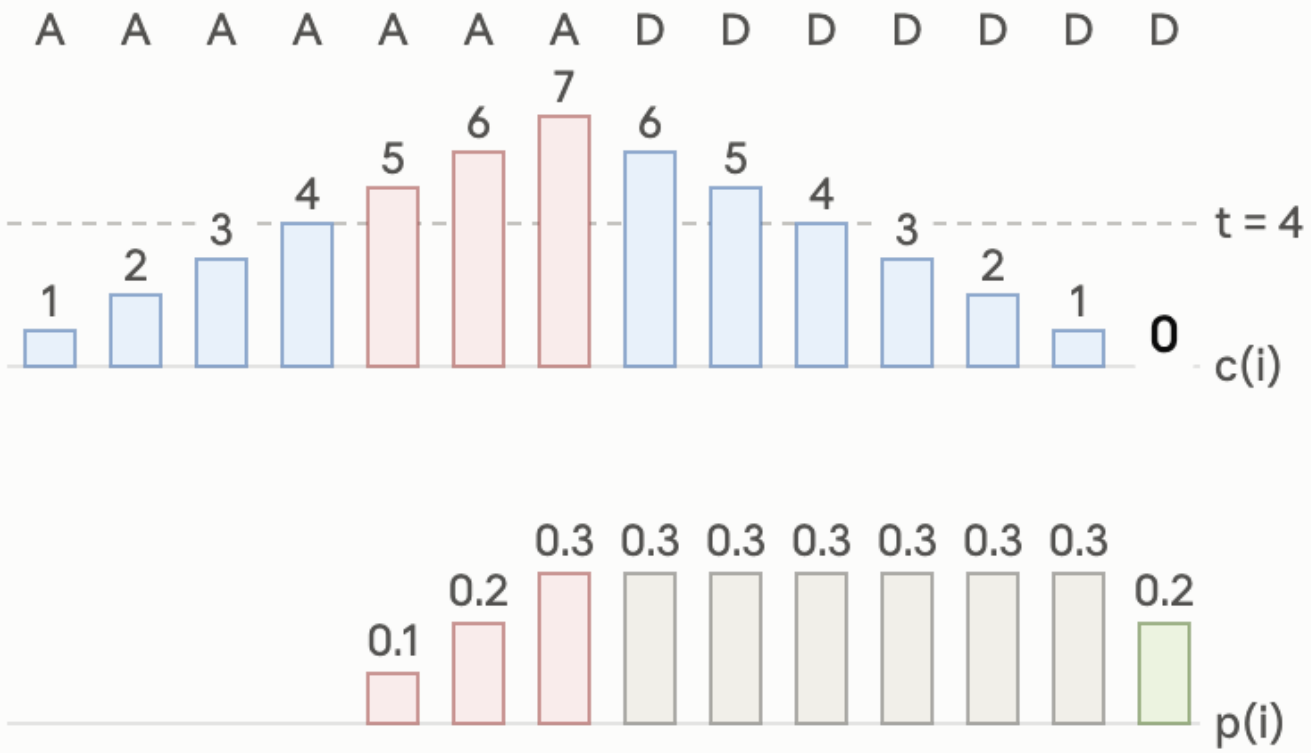
Departure (packet of flow f)

for each bin i in h(f):
c(i) down by 1;
if c(i) = 0, p(i) down by δ

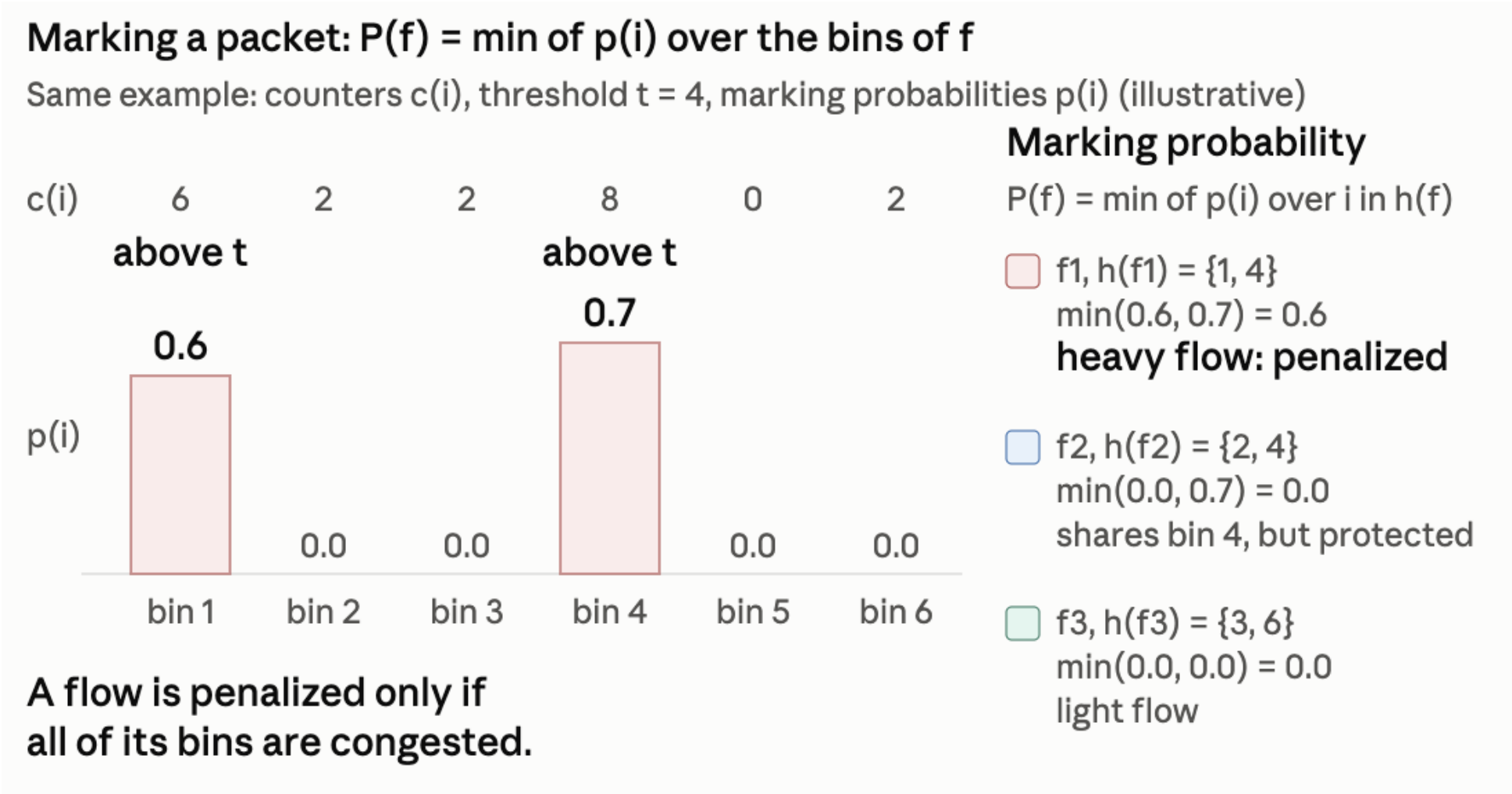
t = threshold, δ = step
p(i) stays within [0, 1]

A = arrival, D = departure
Red: p rises (arrival, c(i) > t)
Green: p falls (c(i) reaches 0)

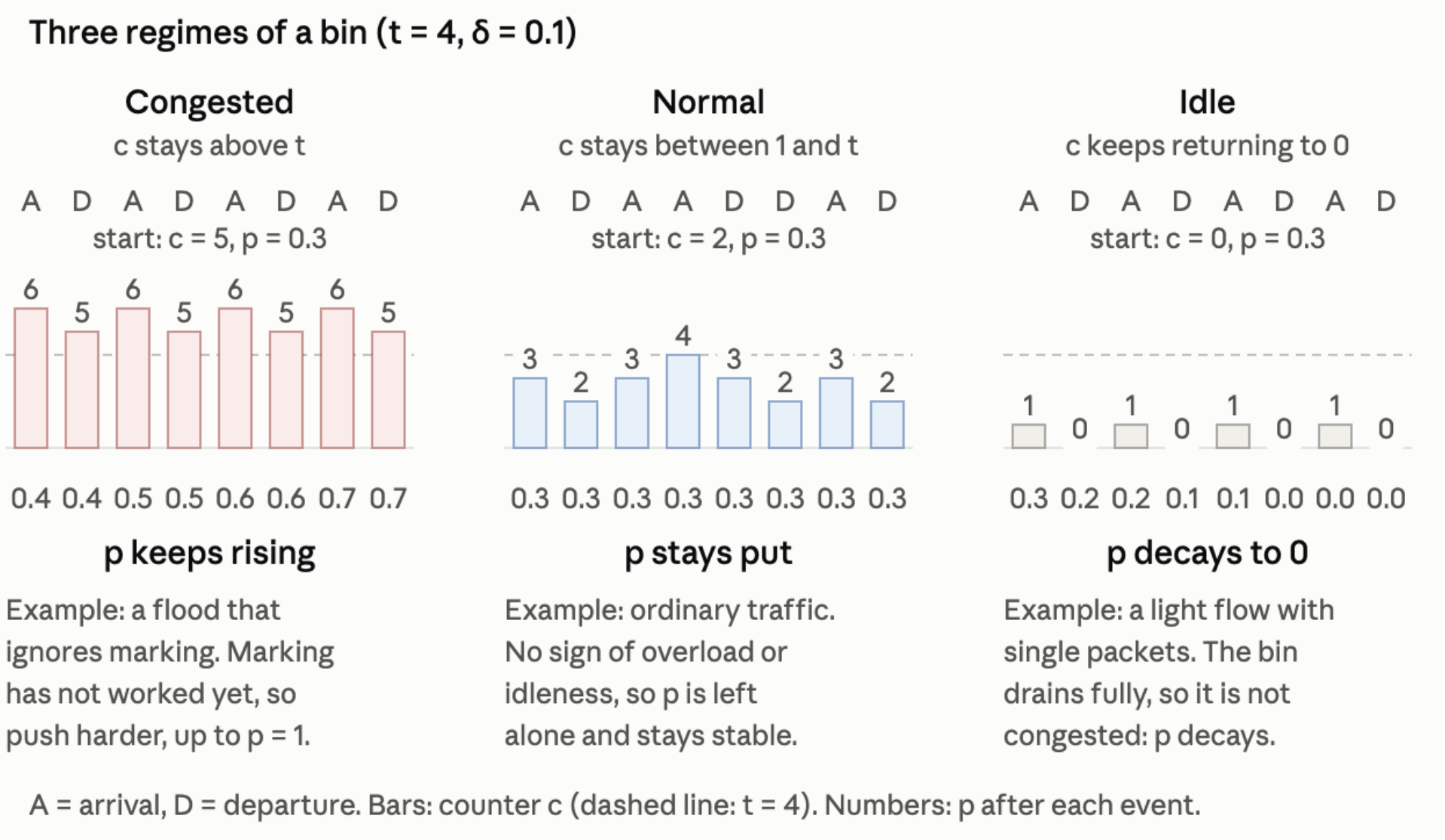
One bin i over time (t = 4, δ = 0.1)



How to mark a packet



Understand a single bin



What we learned

- A Bloom filter can hold a policy, not just a set. Counters and marking probabilities in m bins enforce fairness among millions of flows, with no per-flow state. The cost per packet is k hash operations and k counter accesses, and the answers are approximate.
- The structure is a feedback loop. Marking slows down a heavy flow (if it reacts to losses, as TCP does), its bins drain, and $p(i)$ comes back down. A flow that ignores marking keeps pushing $p(i)$ up to the cap of 1. The scheme adapts to how senders behave, without knowing their rates.
- Errors are cheap here. A collision can only make an innocent flow look guilty if all of its k bins are congested at once, which is rare. Even then the cost is an occasional extra discard, not a wrong answer. That is why an approximate structure with collisions is good enough, as in many Bloom filter applications.